

MuTau Channel: proton mixing & weight normalization issues

Analysis Status

1.overview

Goal

Search for exclusive di-tau production in the muon-tau channel using 2018 data with the Precision Proton Spectrometer (PPS).

Samples Used

Sample	Description	Source
Data	2018 UL SingleMuon	Real collision data
DY	Drell-Yan Z/gamma* to tau-tau	MC simulation
ttbar	tt to tau-tau + X	MC simulation
QCD	Multi-jet background	Data-driven (same-sign)
Signal	Exclusive →	MC simulation

2.Processing pipeline

NanoAOD (CMS)	Phase 0 Skimming	Phase 1 Selection	Merge Files
------------------	---------------------	----------------------	----------------

Plots	Proton Mixing	Merged Phase 1 Files (DY, ttbar, Data, QCD)
-------	------------------	--

3. Phase0: initial skimming

Scripts

- fase0_data_mutau.py (Data)
- fase0_dy.py (Drell-Yan)
- fase0.py (ttbar)

Structure of scripts

1. **Trigger selection:** HLT_IsoMu24 (single muon trigger)
2. **Luminosity filtering:** Only certified good runs (dadosluminosidade.txt)
3. **Object selection:** At least 1 muon and 1 tau
4. **Variable extraction:**
 - Muon: pT, η , ϕ , mass, charge, ID
 - Tau: pT, η , ϕ , mass, charge, DeepTau IDs
 - Jets: pT, η , ϕ , mass, b-tag
 - MET: pT,

Proton variables saved (data only) $\rightarrow$ for MC samples these are added in with the proton mixing

```
nproton_multi, nproton_single
proton_multi_xi, proton_multi_arm, proton_multi_t
proton_multi_thetaX, proton_multi_thetaY
proton_multi_time, proton_multi_timeUnc
proton_single_xi
```

output

- ROOT files with tree containing selected events
 - weight = 1.0, generator_weight = 1.0 (for Data)
-

4. phase1

scripts

- fase1_data.py (Data)
- fase1_dy.py (Drell-Yan)
- fase1_ttjets.py (ttbar)
- fase1_qcd.py (QCD)

Selection cuts applied

Variable	Cut	Description
muon_id	> 3	Medium MVA ID
tau_id1 (VSjet)	> 63	VeryTight DeepTau vs jets
tau_id2 (VSe)	> 7	Tight DeepTau vs electrons
tau_id3 (VSmu)	> 1	Loose DeepTau vs muons
muon_pt	> 35 GeV	muon transverse momentum
tau_pt	> 100 GeV	tau transverse momentum
muon_charge $\times$ tau_charge	< 0	opposite sign
$\Delta R(,)$	> 0.4	Angular separation

Variable	Cut	Description
	< 2.4	geometrical acceptance

Note: QCD uses **same-sign** (SS) selection

5.event weights in phase1

data

```
event_weight = 1.0  #Real data no correction needed
```

Drell-Yan(DY)

```
event_weight = generator_weight × mu_trig_sf × mu_idiso_sf × mu_reco_sf
```

Where: - **generator_weight**: MC generator weight (NLO corrections, etc.) -
mu_trig_sf: Muon trigger scale factor from POG - **mu_idiso_sf**: Muon ID
and isolation scale factor - **mu_reco_sf**: Muon reconstruction scale factor

NOTE: Each SF is calculated with specific function present in the phase1 script

ttbar

```
event_weight = 0.15 × mu_trig_sf × mu_idiso_sf × mu_reco_sf
```

NOTE: Each SF is calculated with specific function present in the phase1 script
(same structure as DY)

Important: The cross-section normalization factor (0.15) is **already included**
at this stage → **this should be fixed, so the analysis steps are the same**
across samples, but it is not a priority now

QCD

```
event_weight = 1.0  #data driven
```

6.Proton Pool

- collection of real protons from 2018 PPS data, used to simulate pileup protons in MC.

Location

/eos/cms/store/group/phys_smp/Exclusive_DiTau/proton_pool_2018/proton_pool_2018.root

Contents

Branch	Description
proton_arm	detector arm
proton_xi	fractional momentum loss = $(p_{\text{beam}} - p_{\text{proton}})/p_{\text{beam}}$

How it was created (script also available in git repo- this code was kept as was)

1. Start with 2018 data with signal triggers
2. no kinematic cuts are applied on the central system
3. protons detected from PPS are extracted
4. we get the final sample with random pileup protons

how it works

- Protons have distribution for 2018 running conditions
- includes detector acceptance and efficiency effects
- random sampling gives realistic pileup simulation

7.Proton Mixing Algorithm

- MC samples (DY, ttbar) don't have reliable pileup protons. they need to be added.

Script

merge_pp_mutau.py

Step 1: Always assign 1 proton per arm

```
#loops through proton pool until we find:  
# - 1 proton with arm=0 -> stored as xi_arm1_1  
# - 1 proton with arm=1 -> stored as xi_arm2_1
```

Step 2: Probabilistically add 2nd protons

variable name	probability	description
P	8.0%	1 proton on arm0, 1 proton on arm1
P	2.0%	1 proton on arm0, 2 protons on arm1
P	2.0%	2 protons on arm0, 1 proton on arm1
P	0.5%	2 protons on each arm

Output branches

Branch	Description
xi_arm1_1	1st proton on arm 0 (always filled)
xi_arm1_2	2nd proton on arm 0 (-1 if none)
xi_arm2_1	1st proton on arm 1 (always filled)
xi_arm2_2	2nd proton on arm 1 (-1 if none)

8.proton acceptance

- The probability that a random event has **at least 1 proton detected on both arms** simultaneously.

Why?

- after the proton mixing code, 100% of the mc events have 1 or more protons per arm
- this does not reflect number of pileup protons in real data samples
- we use a weight to account for the fraction of events that dont have pileup protons detected

Measurement from Data

Sample: Data_2018_UL_MuTau_nano_merged_proton_vars.root

Total events: 6,970

Events with 1 or more proton on both arms: 1,708

$P = 1,708 / 6,970 = 0.245$

$P = 24.5\%$

Category	Events	Fraction
no protons on either arm	1,849	26.5%
Protons only on arm 0	1,600	23.0%
Protons only on arm 1	1,813	26.0%
Protons on both arms	1,708	24.5%

- Matteo's original measurement: **P = 0.13 (13%)**
- doesnt match with the current value

needed: which value is correct for this analysis?

9. Bug in code -> pileup proton mixing

Original Code

```
# In merge_pp_mutau.py, line 143-144:
weight = array("d", [float(FIXED_WEIGHT)])
t_out.Branch("weight", weight, "weight/D")

# In event loop:
# weight was never updated -> was hard coded to 0.13
```

Problem: The original event_weight was discarded

Fixed code (corrected version)

```
#reads input event_weight if it exists
has_event_weight = ensure_branch(t_in, "event_weight")
if has_event_weight:
    event_weight_in = array("d", [1.0])
    t_in.SetBranchAddress("event_weight", event_weight_in)

# In event loop:
if event_weight_in is not None:
    weight[0] = event_weight_in[0] * FIXED_WEIGHT
else:
    weight[0] = FIXED_WEIGHT
```

Now: $\text{weight} = \text{event_weight} \times 0.13$ (preserves all original events' weight factor).

10. Where event weights are applied and values saved

data (no proton mixing needed)

```
Phase0: weight=1.0
Phase1: weight=1.0
Plotting: final_weight=1.0
```

Drell-Yan

```
Phase1: event_weight=generator_weight * muon_SF
Proton Mixing: weight=event_weight * 0.13
Plotting:
final_weight=weight*1.81 (cross-section normalization)
```

ttbar

```
Phase1: event_weight=0.15*muon_SF<-0.15 included in earlier step
```

Proton Mixing: $\text{weight} = \text{event_weight} \times 0.13$
 Plotting:
 $\text{final_weight} = \text{weight} \times 1.0 \rightarrow$ no additional factor needed

QCD (data-driven)

Phase1: $\text{weight}=1.0$ (same-sign selection)
 Plotting: $\text{final_weight}=1.0$

11. Cross-Section normalization factors

formula used

$\text{weight} = N_{\text{MC}}/N_{\text{expected}}$
 where $N_{\text{expected}} = \text{Luminosity} \times \text{Cross-section}$

values used

Sample	Factor	Included Where?
DY	1.81	Applied in plotting
ttbar	0.15	Already in Phase 1 event_weight

$\rightarrow$ this (in principle) does not change the outcome but should be fixed later

12. status of plotting code

file: plot_m.py

how weights are currently used:

```
# QCD: fixed weight
w_qcd=1.0
# DY: reads weight from tree (currently contains only 0.13)
w_dy = get_value(event, "weight", default=1.81)
# ttbar: reads weight from tree (currently contains only 0.13)
w_ttjets = get_value(event, "weight", default=0.15)
```

issues:

1. DY and ttbar files have **weight** = 0.13 (not including event_weight $\rightarrow$ explained earlier)
2. DY needs additional 1.81 factor not being applied
3. ttbar has 0.15 in event_weight but it's lost in current merged files

13.summary of issues found

Issue	Status	Action Required
proton mixing overwrites weights	code fixed	re-run proton mixing
DY cross-section factor missing	pending	apply 1.81 in plotting
proton acceptance value unclear	pending	decide: 0.13 or 0.245
ttbar event_weight lost	code fixed	re-run proton mixing

14.next steps

essential

1. decide on proton acceptance value (0.13 or 0.245)
2. **Re-run proton mixing** for DY and ttbar with fixed script
3. **Update plotting code** to apply correct normalizations

Verification

4. **generate comparison plots** before/after fix
-

15.Files

Processing scripts

Script	Purpose
fase0_data_mutau.py	Phase0 for Data
fase0_dy.py	Phase0 for DY
fase1_data.py	Phase1 for Data
fase1_dy.py	Phase1 for DY
fase1_ttjets.py	Phase1 for ttbar
fase1_qcd.py	Phase1 for QCD

Script	Purpose
merge_pp_mutau.py	Proton mixing
plot_m.py	Plotting

Output samples

File	Description
Data_2018_UL_MuTau_nano_merged_proton_vars.root	merged proton vars
DY_2018_UL_MuTau_nano_good_pileup_protons.root	good pileup protons
ttjets_2018_UL_MuTau_nano_merged_protons.root	merged protons
QCD_2018_UL_MuTau_nano_merged_protons.root	merged protons

16.Proton acceptance measurement code

method in ROOT:

```
TFile* f = TFile::Open("Data_2018_UL_MuTau_nano_merged_proton_vars.root");
TTree* tree=(TTree*)f->Get("tree");
//count total events
Long64_t total = tree->GetEntries();

//count events with one or more proton on both arms
Long64_t good=tree->GetEntries(
    "Sum$(proton_multi_arm==0)>=1 && Sum$(proton_multi_arm==1)>=1"
);

//calculate acceptance
double P = (double)good / (double)total;
cout << "Proton acceptance P = " << P << endl;
```

result from my measurement:

```
total events:6,970
good events:1,708
P=0.245(24.5%)
```

questions

1. Which proton acceptance value should be used?
 - Matteo's 0.13 or measured 0.245?
2. Is the cross-section normalization correct?

- DY: 1.81
 - ttbar: 0.15 (already in event_weight)
-